NYU Tandon School of Engineering
Computer Science and Engineering
CS6083, Spring 2026

April 7, 2026

CS 6083 – Midterm Exam

Instructions. This is a 135-minute exam, with a total of 46 points available.

- Do not open this test booklet until you are told to do so.
- Write your name and student ID on this cover sheet.
- Write your name *at the bottom of each page.*
- Please write your answers neatly *in the box that is provided directly below each question.*
- If you run out of space in a box, use one of the overflow boxes on the last two pages. Put some label inside the original box indicating which overflow box you are using for this question.
- **Do not** write on the margins or the backs of pages.
- Use a pen, not a pencil, if at all possible. If using a pencil, make sure your writing is dark enough to be scanned, and legible.
- The exam is open book, open notes. No electronic devices of any kind may be used, including calculators and electronic watches. Make sure to switch off your phones, and place them and any other devices in your bag.
- Place your ID next to you on your table.
- Good luck!

Name: Sample Solution

Student ID: _____

Problem 1. (24 Points) Suppose you have a database modeling users and messages in Twitter, given by the following schema:

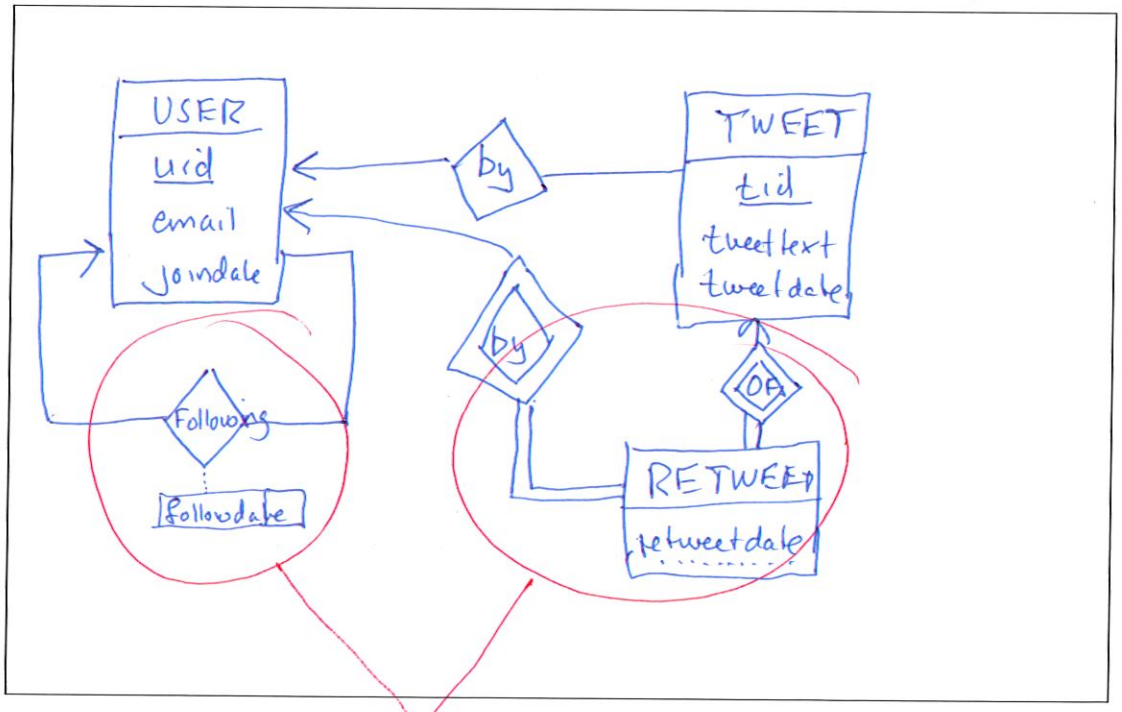
```
USER(uid, uname, email, joindate)
FOLLOWING(followeruid, leaderuid, followdate)
TWEET(tid, uid, tweettext, tweetdate)
RETWEET(tid, uid, retweetdate)
```

In this schema, users have a unique ID, a user name, an email address, and a date and time when they signed up. Users can follow other users, which means that they receive all messages (tweets) that are sent by that person. Tweets have a unique ID, and we also store the ID of the user sending it, the text of the message, and the time and date of the tweet. A user A can also re-tweet a message sent by another user B, in which case the tweet will be forwarded to all followers of A. A tweet may be re-tweeted by many different users, including by users who received it themselves via re-tweets. Note that any re-tweet refers to the original tweet, even if the user only saw the tweet via another person retweeting it. For simnplicity, we assume that users who follow another user cannot unfollow them.

(a) (2 points) Identify suitable foreign keys in the above schema.

followeruid and leaderuid ref User.uid
Tweet.uid refs User.uid
Retweet.tid refs Tweet.tid
Retweet.uid refs User.uid

(b) (4 points) Draw an ER diagram that is consistent with the above schema. Identify cardinalities and any weak entities. If necessary, explain any assumptions you are making.



Note: Following and Retweet may be modeled either as Relationships or as weak Entities!

Problem 1 (c):

(i)
```
SELECT T.tid
FROM Tweet T, Retweet R1, Retweet R2
WHERE  T.tid = R.tid AND T.tid = R2.tid AND
       R1.uid != R2.uid AND R1.uid != T.uid AND
       R2.uid != T.uid
```

(ii)
```
SELECT U.uid, U.uname
FROM User U, Following F1, Following F2, Following F3
WHERE U.uid = F1.followeruid AND
F1.leaderuid = F2.followeruid AND
F2.leaderuid = F3.followeruid AND
F3.leaderuid = F1.followeruid
```

(iii)
```
SELECT T.tid
FROM tweet T JOIN Retweet R ON T.tid = R.tid
      LEFTOUTERJOIN Following F ON
         (R.uid = F.followeruid AND T.uid = F.leaderuid)
WHERE F.followdate = NULL OR F.followdate > R.retweetdate
```

(iv)
```
SELECT U.uid, U.name, count(*)
FROM User U LEFTOUTERJOIN Following F on U.uid = F.leaderuid
GROUP BY U.uid, U.name
ORDER BY count(*) DESC
```

Problem 1 (d): We use P for projection, S for selection, G for group by, and X for crossproduct.
We allow rename when we introduce a table.

```
(i) P_T.tid ( S_{T.tid = R1.tid AND T.tid = R2.tid AND
            R1.uid != R2.uid AND R1.uid != T.uid AND
            R2.uid != T.uid} (Tweet T X Retweet R1 X Retweet R2))

(ii) P_{U.uid, U.uname} ( S_{U.uid = F1.followeruid AND
    F1.leaderuid = F2.followeruid AND
    F2.leaderuid = F3.followeruid AND F3.leaderuid = F1.followeruid}
    (User U X Following F1 X Following F2 X Following F3))

(iii) P_T.tid ( S_{F.followdate = NULL OR F.followdate > R.retweetdate}
     (tweet T JOIN Retweet R ON T.tid = R.tid LEFTOUTERJOIN
      Following F ON (R.uid = F.followeruid AND T.uid = F.leaderuid)))
```

Problem 1 (e): We use EX for "exists", ALL for "for all", and IN for "element of".

```
(i) { S |  EX T in Tweet ( S[tid] = T[tid] AND EX R1, R2 in Retweet (
            T[tid] = R1.tid AND T[tid] = R2[tid] AND
             R1[uid] != R2[uid] AND R1[uid] != T[uid] AND
             R2[uid] != T[uid] ) ) }

(ii) { S | EX U in User ( S[uid] = U[uid] AND S[uname] = U[uname]
         AND EX F1, F2, F3 in Following (
         U[uid] = F1[followeruid] AND F1[leaderuid] = F2[followeruid]
         AND F2[leaderuid] = F3[followeruid]
         AND F3[leaderuid] = F1[followeruid] ) ) }
```

Problem 2. (22 Points)

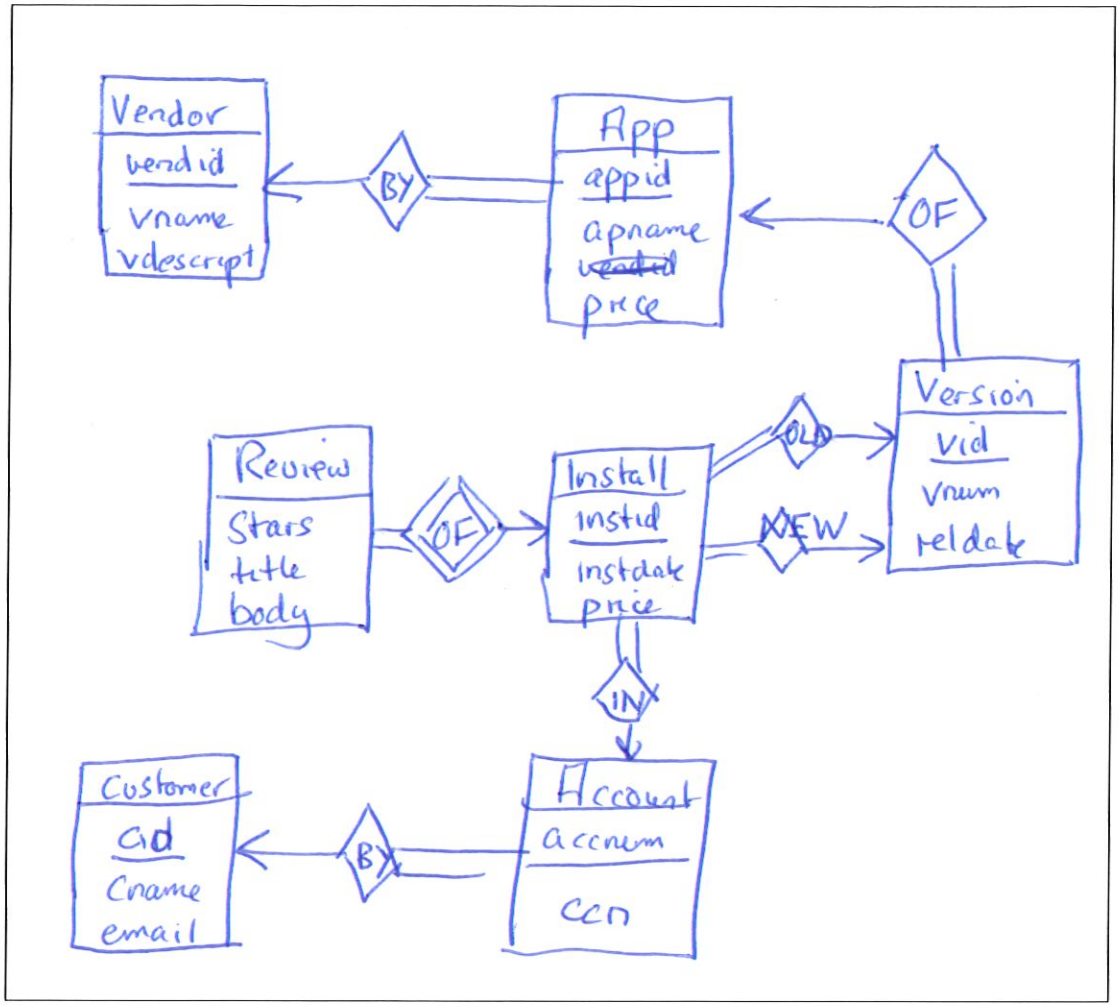
You are asked to design a database for an app store that allows software vendors to sell apps, and that allows customers to install apps. The database should keep track of vendors, customers, apps, and versions of apps, should store which customer account has which version of an app installed, and should also allow customers to review apps.

Of course, we need to simplify the scenario. We assume that each customer has one or more accounts into which they can install apps. We do not model the different devices that a customer may have, or which OS version they run, or which OS version is required by a certain version of the app. We also do not store the actual app executable in the database. So apps get installed into, or added to, a customer account – how they move from their account to their devices is not modeled.

A customer will have information such as a name and an email. An account belongs to a customer, and has a credit card number associated with it – you do not have to model billing address, expiration date, etc. A customer can have several accounts. Each app vendor has a single account, and is identified by maybe an ID and a name. App vendors typically sell a number of different apps, and each app may have many versions (releases), such as 1.1, 2.0, and so on. We assume that each app has a current price that may change over time but is independent of the version that is being installed, and that upgrades from one version to another are free. We do not model free versus paid versions of the same app, and do not model in-app purchases.

When a customer installs or upgrades an app in their account, this should be stored with suitable information such as the previous and new version of the app, the time and date, and the amount charged to their credit card if it is a new app. There are no downgrades to older versions, and we do not assume that any automatic upgrades happen. We do not model how the money gets paid to the vendor, and what the split between vendor and app store is. We do not model deletions of apps. Finally, customers can add reviews for apps they have installed in their accounts – each review has a star rating between one and five stars, a review title and the review body. Reviews are associated with a particular account and a particular version of the app that the user had installed when they did the review, so they can be seen as reviews of the app and of the particular version. (A customer with several accounts could potentially review the same app version several times, though for each account there can be at most one review for each version of an app.)

(a) (8 points) Design an ER schema that can model the above scenario. Identify any weak entities, suitable keys, and the cardinalities of the relationships.



Discuss any assumptions that you are making in your design. (Recommended, but not required if you think your schema is obvious.)

(b) (8 points) Convert your ER diagram into a relational schema. Show primary keys and foreign key constraints.

Problem 2 (b):

Vendor (vendid, vname, vdescript) PK: vendid
Customer (cid, cname, cemail) PK: cid
Account (accnum, cid, ccn) PK: accnum FK: cid refs Customer
App (appid, apname, vendid, price) PK: appid FK: vendid refs Vendor
Version (vid, appid, vnum, reldate) PK: vid FK: appid refs App
Install (instid, vidold, vidnew, accnum, instdate, price)
  PK: instid FK: vidold, vidnew ref Version, accnum refs Account
  If vidold is NULL, this is a new install and price can be > 0
Review (instid, stars, title, body) PK: instid FK: instid refs Install

Problem 2 (c):

(i)
```
SELECT C.cid, C.cname
  FROM Customer C JOIN Account A JOIN Install JOIN App A JOIN Vendor V
  WHERE V.vname = "Bingo Software"
  GROUP BY C.cid
  HAVING count(distinct A.appid) >= 10
```

(ii)
```
SELECT VE.vendid, VE.vname
  FROM Vendor VE JOIN
    ( ( SELECT A.vendid, A.appid
        FROM  App A JOIN Version V JOIN Install I
        GROUP BY A.vendid, A.appid
        HAVING count(distinct I.accnum) >= 1000 )
      INTERSECT
      ( SELECT A.vendid, A.appid
        FROM  App A JOIN Version V JOIN Install I JOIN Review R
        GROUP BY A.vendid, A.appid
        HAVING avg(stars) < 2 )
    )
```

(iii)
```
SELECT C.cid, C.cname
  FROM Customer C JOIN Account A1 JOIN Install I1 JOIN Review R1,
         Account A2 JOIN Install I2 JOIN Review R2
  WHERE  I1.vidold = I2.vidnew AND A1.accnum = A2.accnum AND
           R1.stars <= R2.stars - 2
```